AI POWERED STUDENT STRESS PREDICTION AND WELLNESS RECOMMENDATION SYSTEM

STUDENT NAME: BOOMIKA K

REG NO: 922524243022

1. Introduction

In contemporary academic environments, students face unprecedented levels of stress stemming from rigorous coursework, examinations, social obligations, and career uncertainties. Prolonged and unmanaged stress can lead to severe mental health challenges, decreased academic performance, and overall diminished well-being. Traditional methods of detecting and addressing student stress are often reactive, occurring only after noticeable behavioral or psychological decline.

The **AI Powered Student Stress Prediction and Wellness Recommendation System** is an advanced, proactive solution designed to tackle this challenge. By leveraging the power of modern Machine Learning algorithms and data science, the system assesses various physiological, psychological, environmental, and academic parameters to predict an individual's stress level with precision.

Moving beyond mere prediction, the core strength of this system lies in its personalized **Wellness Recommendation Engine**. Depending on the predicted stress status—ranging from low to critical—the system dynamically prescribes actionable, tailored coping strategies, mindfulness routines, and lifestyle adjustments. This integrated approach bridges the gap between diagnostic analytical models and therapeutic care, creating an automated support framework that empowers students to monitor, understand, and safely manage their mental health journey.

2. Objectives

The foundational goal of this project is to develop an intelligent ecosystem that acts as an automated advisor for student mental wellness. The comprehensive objectives include:

- **Accurate Stress Level Prediction:** To construct and train high-performance machine learning classification algorithms capable of correctly categorizing student stress into distinct, actionable tiers (Low, Moderate, High, and Critical).
- **Dynamic Recommendation Engine:** To architect an intelligent wellness system that analyzes real-time predictions and maps them to tailored behavioral advice, lifestyle modifications, time-management tools, and medical guidance.
- **Proactive Self-Monitoring Dashboard:** To deliver an intuitive, confidential user dashboard enabling students to systematically log their data, view immediate results, track historical trends, and visually inspect their longitudinal stress indicators.
- **Robust Data Pipeline and Secure History Maintenance:** To establish a secure relational database architecture to safely preserve encrypted historical user inputs and diagnostic outcomes, ensuring complete data privacy while facilitating continuous user self-tracking.

3. System Modules

The implementation framework is structured into interconnected core modules, each managing a specific segment of the application lifestyle:

3.1 Data Collection & Preprocessing Module

This module oversees the aggregation of student metrics across various dimensions (such as sleep hours, study load, extra-curricular stress, and physical activity). It performs foundational preprocessing, including missing value imputation, handling outliers, and performing min-max scaling to format inputs uniformly for model consumption.

3.2 Machine Learning Model Training Module

Responsible for managing historical baseline datasets, splitting data into training/testing distributions, and training predictive models (such as Random Forest or Gradient Boosting classifiers). The optimal model configuration is serialized and stored via model-persistence libraries for real-time deployment.

3.3 Real-time Prediction Engine

Acts as the production pipeline. It captures active inputs submitted by a student through the web application, performs instantaneous forward-pass transformations, and yields an evaluated categorical stress outcome.

3.4 Personalized Wellness Recommendation Engine

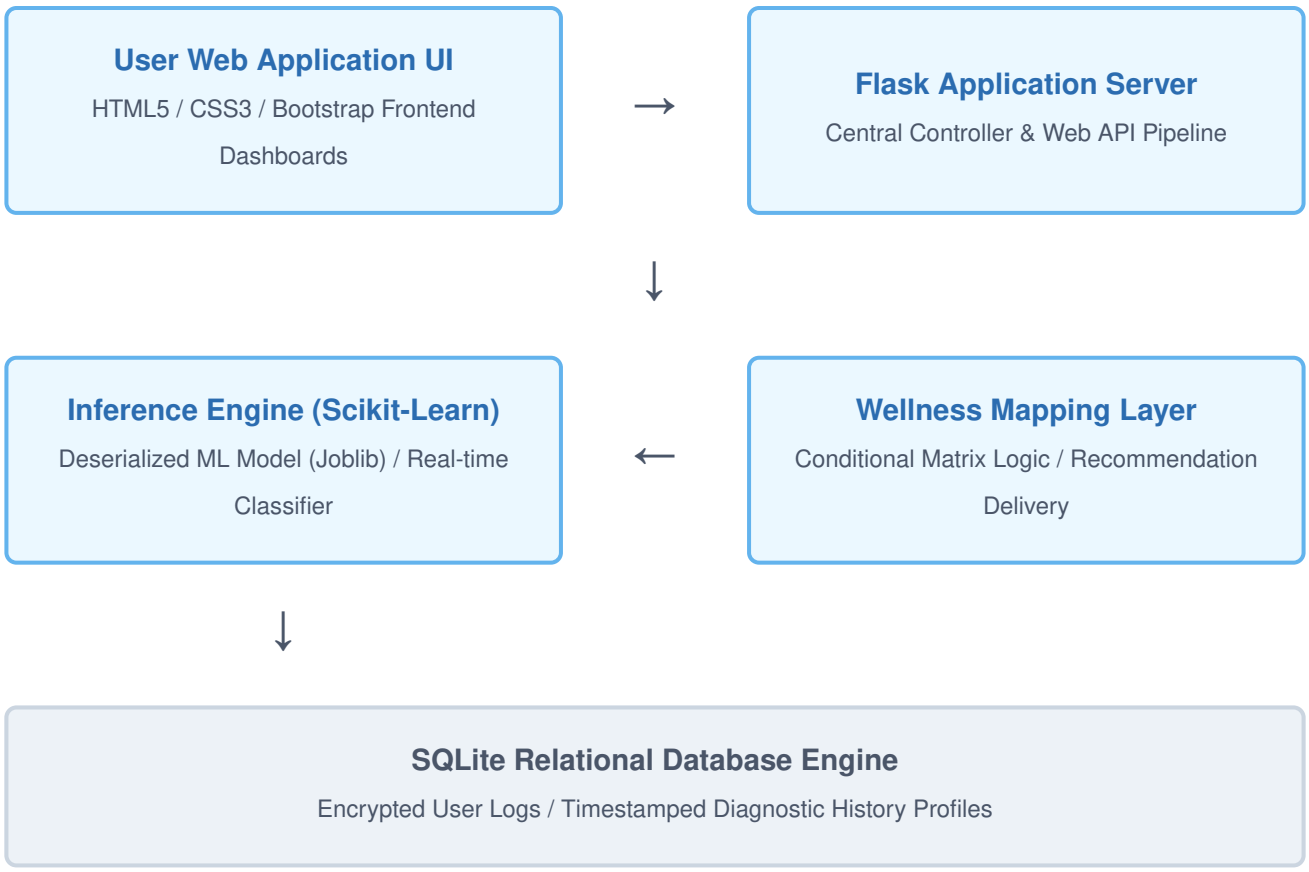
Evaluates the classified stress output and triggers customized wellness logs. It renders cognitive behavioral tips for moderate stress, intensive relaxation/scheduling frameworks for high stress, and urgent therapeutic alerts for critical tiers.

3.5 Relational Database & Dashboard Module

Manages secure application data persistence using SQLite. It records student profile details alongside timestamped logs of stress results, feeding an interactive analytics frontend crafted using HTML5, CSS3, and Bootstrap components.

4. System Architecture

The structural blueprint of the system follows a classic N-Tier model-view-controller web topology integrated with a decoupled inference pipeline. The architecture diagram below illustrates the linear flow of data from the user application interface to the persistent backend storage:



As depicted, the user safely inputs academic and physiological features through the dashboard. The request hits the Flask routing controller, which invokes the saved ML serialized instance. The prediction output is evaluated by the logic module to dynamically extract appropriate support recommendations. Finally, both the results and diagnostic payloads are persisted into the SQLite relational layer before formatting the dashboard view for the student.

5. Implementation Coding

The application is fully developed using Python and its corresponding data engineering ecosystem. Below is the clean implementation structure showcasing model loading, web request routing, stress prediction execution, and dynamic wellness mapping:

```
import flask
from flask import Flask, render_template, request, redirect, url_for
import joblib
import pandas as pd
import sqlite3
from datetime import datetime

app = Flask(__name__)

# Load the trained Machine Learning serialization file
model = joblib.load('models/stress_predictor_model.pkl')

def get_wellness_recommendations(stress_level):
    if stress_level == "Low Stress":
        return "You're doing great! Maintain your routine, stay active, and practice
continuous learning."
    elif stress_level == "Moderate Stress":
        return "Keep going, you can do better! Take short breaks, organize tasks, and
stay focused."
    elif stress_level == "High Stress":
        return "Take a step back and relax! Prioritize sleep, reduce project loads,
and practice mindfulness."
    elif stress_level == "Critical Stress":
        return "It's critical to seek support immediately! Reach out to counselors,
cut down work, and seek help."
    return "Keep monitoring your lifestyle choices regularly."

@app.route('/predict', methods=['POST'])
def predict_stress():
    # Fetch parameters from the user interface form
    sleep_hours = float(request.form['sleep_hours'])
    study_hours = float(request.form['study_hours'])
    extracurricular = float(request.form['extracurricular'])

    # Construct dataframe payload for structural inference
    features = pd.DataFrame([[sleep_hours, study_hours, extracurricular]],
                            columns=['sleep_hours', 'study_hours',
'extracurricular'])
```

```
    # Execute class inference
    prediction_id = model.predict(features)[0]

    # Map predictions to human readable tiers
    stress_mapping = {0: "Low Stress", 1: "Moderate Stress", 2: "High Stress", 3:
"Critical Stress"}
    result_tier = stress_mapping.get(prediction_id, "Unknown Status")

    recommendation = get_wellness_recommendations(result_tier)

    # Log the complete diagnostic outcome into SQLite DB for tracking history
    conn = sqlite3.connect('database/wellness.db')
    cursor = conn.cursor()
    cursor.execute("INSERT INTO logs (timestamp, result, advice) VALUES (?, ?, ?)",
                   (datetime.now().strftime("%Y-%m-%d %H:%M:%S"), result_tier,
recommendation))
    conn.commit()
    conn.close()

    return render_template('dashboard.html', prediction=result_tier,
advice=recommendation)

if __name__ == '__main__':
    app.run(debug=True)
```

6. System Screenshots

The dashboard presents a clear interface that changes color dynamically based on the student's evaluated stress category. Below are the user interface screens representing each stress level outcome:

Figure 6.1: Low Stress Status View

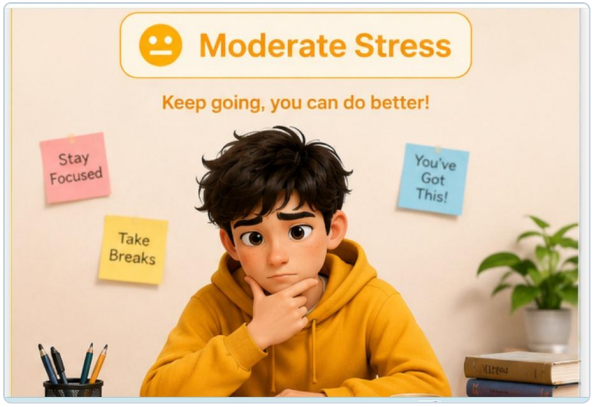


Figure 6.2: Moderate Stress Status View

6. System Screenshots (Contd.)

When a student registers higher levels of study load, sleep deprivation, or pressure, the application interface securely alerts them through higher stress indicators:

Figure 6.3: High Stress Status View

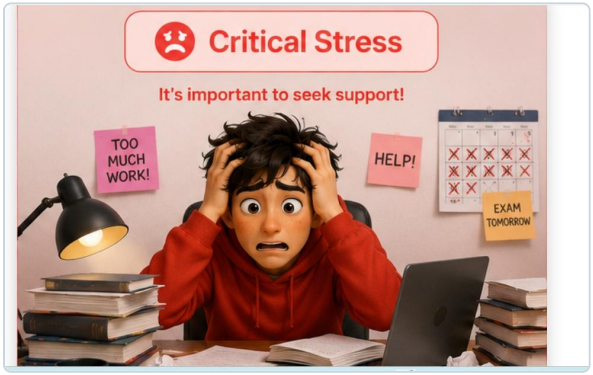


Figure 6.4: Critical Stress Status View